

Power Analysis Attacks on ECC Randomized Automata

Fan Zhang and Zhijie Jerry Shi

Department of Computer Science and Engineering, University of Connecticut, Storrs, CT 06269, USA

Email: {fan.zhang, zshi}@engr.uconn.edu

Abstract- Elliptic curve cryptography (ECC) has been widely used in many systems because it requires shorter keys than traditional public-key algorithms. However, power analysis attack, a type of side channel attacks, can exploit the instantaneous power consumptions of ECC devices and retrieve the secret key used in the computations. Many countermeasures have been proposed to make ECC implementations secure against power analysis attacks. One of the approaches is randomized algorithms that generate different power traces even if the input of the algorithm is the same. For example, Oswald et al. proposed two randomized algorithms that combine two scalar point multiplication algorithms and use random variables to decide which algorithm to follow at different stages of the computation. Previously, we have successfully launched attacks on Randomized Automaton 1 [1]. In this paper, we extend our prior work and present the details of our attack. We also take Randomized Automaton 2 as an example and demonstrate how our attack can retrieve the secret key used in the algorithm.

I. INTRODUCTION

The arithmetic of large integer in finite fields has been adopted in many public-key algorithms, such as RSA and ElGamal, and key-exchange algorithms, such as Diffie-Hellman. In the middle 1980s, Miller and Koblitz proposed the elliptic curve cryptography (ECC) [2][3] as an alternative to large integer arithmetic. ECC has attracted a lot of attention because it can provide equivalent security strengths with shorter keys than traditional large integer operations. Smaller key sizes have many advantages including higher performances, smaller storage space, and lower energy consumptions. These advantages are especially important in resource-constrained environments [4].

Power analysis is a type of side channel attacks that exploit the power consumptions of cryptographic devices to reveal the secret data used in cryptographic computations. There are several types of power analyses, such as Simple Power Analysis (SPA) and Differential Power Analysis (DPA). SPA exploits the correlation between the power outputs and the operations. By observing the power trace of cryptographic computations, adversaries can learn what operations are performed and then figure out part or all of the secret data. SPA can be launched even with only one power trace. Unlike SPA, DPA uses statistical methods to test whether a specific value is generated during cryptographic computations and then deduces the secrets. Typically, DPA needs a large number of power traces. Both of SPA and DPA can be used to attack ECC implementations.

Some countermeasures have been proposed to make ECC systems secure against power analysis. One of the countermeasures is the randomized algorithm proposed by Oswald et al. in [5]. The randomized algorithms seem secure against traditional power analyses. However, we found they are vulnerable to a new type of power analysis attacks and presented our attack on Randomized Automaton 1 in [1]. This paper will give the details of our attack, such as the algorithm that finds unique sequences and the method that removes shortened the power traces when the some key bits are recovered. We will also demonstrate how our method can break Randomized Automaton 2.

The rest of the paper is organized as follows. Section II describes the basic operations of ECC briefly and Section III describes power analysis attacks and some countermeasures. In Section IV, we present our attack and show how it can be used to break Randomized Automaton 2. Section V discusses related work and Section VI concludes the paper.

II. ELLIPTIC CURVE CRYPTOGRAPHY

An elliptic curve is a set of points (x, y) specified by a bivariate cubic equation defined over a field K [6]:

$$y^2 + a_1xy + a_3y = x^3 + a_2x^2 + a_4x + a_6. \quad (1)$$

where a_i ($i \in \{1, 2, 3, 4, 6\}$), x , and $y \in K$. If K is $\text{GF}(2^n)$, equation (1) can be reduced to:

$$y^2 + xy = x^3 + ax^2 + b. \quad (2)$$

where a, b, x , and $y \in \text{GF}(2^n)$.

The set of points on an elliptic curve, together with a special point O called the point at infinity and an addition operation, form an abelian group [7]. For elliptic curves defined over $\text{GF}(2^n)$, the addition operation of points is defined as follows. If a point $P = (x_1, y_1) \neq O$, $-P = (x_1, x_1 + y_1)$. Given another point $Q = (x_2, y_2) \neq O$ and $Q \neq -P$, the sum $(x_3, y_3) = P + Q$ can be computed as:

$$\begin{aligned} x_3 &= \lambda^2 + \lambda + x_1 + x_2 + a. \\ y_3 &= \lambda(x_1 + x_3) + x_3 + y_1. \end{aligned} \quad (3)$$

where

$$\lambda = \begin{cases} \frac{y_2 + y_1}{x_2 + x_1}, & \text{if } P \neq Q \\ \frac{y_1}{x_1} + x_1, & \text{if } P = Q. \end{cases}$$

The multiplication of an integer d and a point P is defined as adding d copies of P together.

$$Q = dP = \underbrace{P + \dots + P}_d$$

III. POWER ANALYSIS AND COUNTERMEASURES

In this section, we briefly describe the power analysis of ECC and some countermeasures [8].

A. Power Analysis of ECC

Scalar point multiplication, $Q = dP$, is the basic operation of ECC. It is equivalent to the exponentiation operation in the multiplicative group of large integers. It can be performed with the binary algorithm BINALG as shown in Fig. 1a. In the figure, l is the number of bits in d and $d_{l-1} = 1$.

In BINALG, point doubling $2Q$ is performed in every iteration while point addition $Q+P$ is performed only for 1 bits. Since point doubling and point addition are performed with different formulae (see Section II) and thus have different power outputs, BINALG is vulnerable to SPA. An adversary can monitor the power consumption of a device performing BINALG and identify the point addition and doubling operations. A doubling followed by an addition indicates a 1 bit in d and two consecutive doublings indicate a 0 bit. The adversary can easily retrieve the value of d , which is supposed to be secret in many algorithms.

To make the algorithm SPA resistant, one can remove the correlations between the execution path and the data being processed. In BINALG, the execution of point additions depends on the value of d_i . The additions are performed only when $d_i = 1$. In order to thwart SPA attacks, Coron [6] proposed BINALG', as shown in Fig. 1b. BINALG' is a variant of BINALG. It always performs the point addition. The correct value is selected by d_i and carried over to following iterations. According to [6], BINALG' is secure against SPAs. However, Coron discovered that it is vulnerable to DPA [6]. Oswald and Aigner further pointed out that any similar algorithms are also vulnerable to DPA attacks [5].

B. Randomized Algorithm

Oswald and Aigner proposed two randomized algorithms trying to thwart both SPA and DPA. In these two algorithms, they combined a variant of BINALG and an algorithm based on addition-subtraction chains [9]. The first algorithm, referred to as Randomized Automaton 1, is illustrated in Fig. 2. The second algorithm, which will be referred to as Randomized Automaton 2, is shown in Fig. 3. When performing a scalar point multiplication dP with either of the automata, we start from State 0 and scan the bits in d from the least significant bit. For every bit, we either advance to a new state or stay in the same state, as specified in the automata, and perform proper operations during transitions.

In both automata, some transitions are decided by random variables and only the transition with the correct random value will be followed. These transitions are illustrated with dashed lines in Fig. 2 and Fig. 3. For example, when the automaton

enters State 1 in Randomized Automaton 1, a random variable e is drawn. If the next bit is 1, the automaton will stay in State 1 if $e = 1$. Otherwise it will go to State 11. Because of the random variables, the randomized scalar point multiplication algorithms follow different paths and thus perform different operations even if the scalar d remains the same.

BINALG
Input : P and d
Output: $Q = dP$
$Q \leftarrow P$
For i from $l-2$ to 0 do
$Q \leftarrow 2Q$
If $d_i = 1$ then $Q \leftarrow Q + P$
Return Q

(a) Binary algorithm

BINALG'
Input: P and d
Output: $Q[0] = dP$
$Q[0] \leftarrow P$
For i from $l-2$ to 0 do
$Q[0] \leftarrow 2Q[0]$
$Q[1] \leftarrow Q[0] + P$
$Q[0] \leftarrow Q[d_i]$
Return $Q[0]$

(b) Double-and-add algorithm

Fig. 1: Scalar point multiplication algorithms

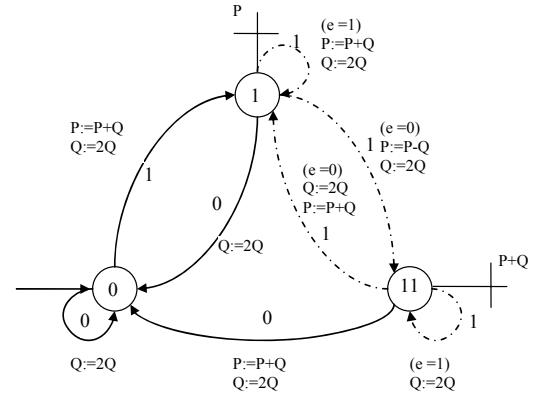


Fig. 2: Randomized Automaton 1 [5]

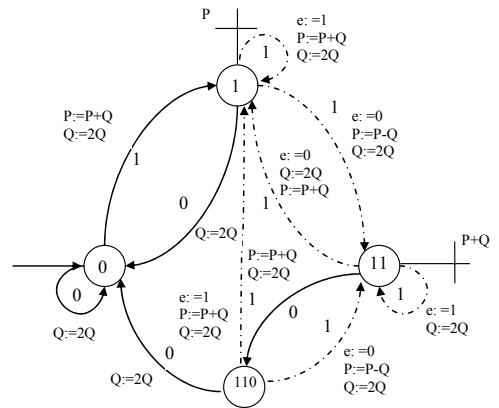


Fig. 3: Randomized Automaton 2 [5]

Randomized Automaton 2 is an improvement of Randomized Automaton 1. It has one more state State 110. In State 110, the automaton goes to State 0 if the next key bit is 0. However, if the key bit is 1, the automaton may go to either State 1 or State 11. Compared with Randomized Automaton 1,

it is more resistant against traditional SPA or DPA attacks because the increased number of the states and random variables makes analyses more complicated.

Okeya and Takagi also proposed a method to attack Randomized Automaton 1 [10]. However their method has not been shown effective on Randomized Automaton 2. In the next section, we will describe our attack and show how it can break Randomized Automaton 2.

IV. ATTACKS ON RANDOMIZED ALGORITHMS

Our attacks to Randomized Automaton 2 are the extended work of our previous attacks to Randomized Automaton 1 [1]. This section describes our attacks in detail.

A. Attack on Randomized Automaton 2

To describe our attack much more clearly, we rename two states in Automaton 2. We will call State 11 as State 2 and State 110 as State 3. The doubling operation $2Q$ will be denoted as D and the addition or subtraction operation $(P+Q$ or $P-Q)$ as A. We assume additions and subtractions are indistinguishable. The bits in scalar d are referred to as *input bits* because they are the input to the automaton during the computation.

To launch the attack, we first collect a set of power traces by performing the scalar point multiplication N times. Although all the power traces start from State 0, they visit different states because of the random variables. When we collect the power traces, we observe the state of automaton after each input bit is processed. The state of automaton can be summarized in four cases. 1) The automaton is in State 0 for all power traces. 2) The automaton is in State 1 for all power traces. 3) The automaton can be in either State 1 or State 2. And 4) the automation can be in either State 0 or State 3. During an attack, we stay in different *attack states* according to the state of automaton. So we have four attack states AS0, AS1, AS1/2 and AS0/3, which correspond to the four cases. For example, in AS1/2, the automaton could be in either State 1 or State 2. The uncertainty is introduced by the random variables. The two cases are evenly distributed among power traces.

Our attack to Randomized Automaton 2 has three main steps: (1) *Detect input bits*. We look at *all* power traces and try to detect one or several input bits a time for the current *attack state* with *unique sequences*. (2) *Commit detected bits*. A bit is *committed* if the bit is appended to the recovered input string and its corresponding power output symbols are removed from power traces. (3) *Go to next attack state*. We decide the next attack state according to the committed bits. After we enter the next attack state, we repeat these steps and detect more input bits with shortened power traces until all input bits are recovered.

B. Detecting Input Bits

We first look at how to detect the input bits from the power traces in each attack state.

We detect the input bits by *unique sequences*. In an attack state, the automaton may start from one or two states. Let us use S to denote the set that includes all possible starting states. For example, in AS0 $S = \{0\}$ and in AS2 $S = \{1, 2\}$. We use I to denote the set that includes the possible input strings we are interested in. We want to identify correct input string in I by examining all the power traces. Starting from a state in S , a string in I may generate different power traces because of the random variables. Given S and I , we can collect a set R which consists of all possible power traces. An element in R is a 3-tuple (s, k, p) which denotes that p is a power trace that can be generated by input string k from state s , where $k \in I$ and $s \in S$. Two elements in R may have the same s and k but different p .

Consider $(s, k, p) \in R$, p is a *unique sequence* if it satisfies the following conditions.

Condition 1: p cannot be generated by another string in I , i.e., if $(s', k', p') \in R$ and $k \neq k'$, then $p \neq p'$.

Condition 2: If $(s', k', p') \in R$ and $k \neq k'$, then p is not a leading substring of p' and p' is not a leading substring of p .

Basically, Condition 2 says that adding or removing A's and D's at the end of p cannot convert it into another sequence p' that can be generated by a different string k' in I .

Fig. 4 gives an algorithm that finds unique sequences in R .

FIND-UNIQUE-SEQUENCE

Input : $R = \{r_1, r_2, \dots, r_n\}$ where $r_i = (s_i, k_i, p_i)$

Output: Unique Sequences Set U

```

1   $U = \{\}$ ;
2  for ( $i=1; i \leq n; i++$ ) {
3      IsUS=true;
4      for ( $j=1; j \leq n; j++$ ) {
5          if ( $k_i \neq k_j$  and ( $\text{Substr}(p_i, p_j)$  or  $\text{Substr}(p_j, p_i)$ )) then {
6              IsUS=false;
7              break;
8          }
9      }
10     if IsUS then {
11          $U = U \cup r_i$ ;
12     }
13 }
```

Fig. 4: Algorithm to find unique sequences

The output of the algorithm U is a subset of R . If $(s, k, p) \in U$, p is a unique sequence for input string k in automaton state s for the input string set I . Note that a string k may have more than one unique sequence and U may have unique sequences for more than one input string.

The algorithm in Fig. 4 checks every element in R . If an element meets the two conditions for unique sequences, it is added into U . Function $\text{Substr}(x, y)$ in Line 5 checks whether x is a leading substring of y , i.e., whether y can be generated from x by appending A's and D's. For example, it returns true when $x = \text{ADD}$ and $y = \text{ADDA}$. Note that we do not explicitly check Condition 1 because $\text{Substr}(x, y)$ returns true when x is the same as y .

TABLE I: AD SEQUENCES OF THREE INPUT BITS IN ALL STATES

	State	Input Bits	Power Output Sequences
1.	0	0xx	D
2.		1xx	AD
3.	1	0xx	D
4.		111	ADDD
5.			ADADD
6.			ADDDA
7.			ADADAD
8.			ADDAAD
9.			ADADDA
10.		110	ADADD
11.			ADDAD
12.			ADADAD
13.		101	ADDAD
14.			ADADAD
15.		100	ADDD
16.			ADADD
17.	2 (11)	111	DDD
18.			DDDA
19.			DDAAD
20.			DAADD
21.			DAADDA
22.			DAADAD
23.		110	DDAD
24.			DAADD
25.			DAADAD
26.		101	DADAD
27.		100	DADD
28.	3 (110)	111	ADDD
29.			ADADD
30.			ADDDA
31.			ADADAD
32.			ADDAAD
33.			ADADDA
34.		110	ADADD
35.			ADDAD
36.			ADADAD
37.		101	ADDAD
38.			ADADAD
39.		100	ADDD
40.			ADADD

With unique sequences found in U , we can identify some input strings in the attack state. We check all the power traces, if any power traces starts with a unique sequence p in U , the corresponding input string k must be the correct input string. Since the power traces are determined by random variables, looking at few power traces, we may not see unique sequence p even if k is the correct key bits. However, if we do not see p in *many* power traces (we discuss how many power traces are needed in Section IV.D), we are confident that k is not the correct input string. We can then exclude k from I and get a small set I' . We then generate a new set R' and find unique sequences in I' . With the unique sequences found in R' , we

can check whether their corresponding input strings are the correct input bits. This process is repeated until the correct input bits are identified. Note that when only one string is in I , all the power output sequences are unique sequences. In real attacks, we can find all unique sequences in advance and confirm or exclude strings in I with these sequences.

We take Randomized Automaton 2 as an example to illustrate how we use the algorithm in Fig. 4 to find the unique sequences for each attack state.

Table I list all the AD sequences of three input bits in all states in Randomized Automaton 2. When the automaton is in the state specified in column 2, it can generate the power output sequences in column 4 on the string in column 3. Columns 2, 3, and 4 in each row form a 3-tuple that can be put in R . Note that for States 2 and 3, we are only interested in input strings that start with 1.

To find unique sequences, we first determine S and I . S contains all possible automaton states in an attack state. I contains all input strings listed in Table I initially and becomes smaller as input strings with unique sequences are removed from the set. From S and I , we create R from the rows where the state is in S and the input string is in I . After R is determined, the unique sequences can be found with the algorithm in Fig. 4. If not all strings in I have unique sequences, we can remove from I the strings that have already got a unique sequence and repeat the process. For each attack state, we find unique sequences for all input strings we are interested in. Table II lists the unique sequences for Randomized Automaton 2. If a string is the only one left in I , its unique sequences are not shown in the table because all of its output sequences are unique sequences.

TABLE II: UNIQUE SEQUENCES FOR RANDOMIZED AUTOMATON 2

Attack state	Input Bits	Unique Sequences
AS0	0	D
	1	AD
AS1	0xx	D
	111	ADDAAD
	100	ADDD
	110	ADADD
	101	
AS1/2	111	ADDAAD,DDAAD,DDDA,DDD
	100	DADD
	110	DDAD
	101	
AS0/3	111	ADDAAD
	100	ADDD
	110	ADADD
	101	

The cases in AS0 are trivial. All the power traces are in State 0, so $S = \{0\}$. We choose $I = \{0, 1\}$ because a single 0 or 1 already has different power outputs. So R has two elements from row 1 and row 2 of Table I. Using the algorithm in Fig. 4,

we find the unique sequence D for 0 and AD for 1. Therefore, if we see A is the first operation in the power traces, the input bit must be 1. If we see D, the input bit is 0.

The cases in AS1 are more complicated. In AS1, all the power traces are in State 1. So we have $S = \{1\}$. Let $I = \{0, 1\}$. We can find D is the unique sequence for bit 0 and AD for bit 1. However, when the input bit is 1, simply committing this bit may cause problems. With the input bit 1, we will enter AS1/2 next, in which we cannot find unique sequences for any input strings. To solve the problem, we would like to detect more bits here but do not commit all detected bits. We withhold some detected bits to help us detect input bits in following attack states. Therefore, if the next bit is 1 in AS1, we go further to detect the next three bits. Thus $I = \{111, 100, 110, 101\}$ and R consists of all the tuples generated from row 4 to row 16 in Table I. We find the unique sequence ADDAAD for 111. It means that if we see ADDAAD in any of the power traces, the next three input bits are 111. If it is not the case, we remove 111 from I and continue to identify input bits in the remaining strings. After 111 is excluded, $I = \{100, 110, 101\}$ and R contains the 3-tuples from row 10 to row 16. We can find the unique sequence ADDD for 100. If we do not see ADDD in all power traces, we exclude 100 as well. With $I = \{110, 101\}$, we find the unique sequence ADADD for 110. If 110 is not the correct input bits either, 101 must be because it is the only string left. All the output sequences that can be generated by 101 are unique sequences.

In AS1/2, $S = \{1, 2\}$. We detect three inputs bits a time. And we also make sure we have already withheld at least a 1 bit when we get into AS1/2. So we focus only on the strings that start with 1. So $I = \{111, 100, 110, 101\}$ and R can be constructed from row 4 to row 27 in Table I. Using the algorithm in Fig. 4, we find four unique sequences for 111: ADDAAD, DDAAD, DDDA, and DDD. ADDAAD is a unique sequence that may be generated by the power traces starting from State 1. DDAAD, DDDA, and DDD may be generated by power traces starting from State 2. At the same time, we also find the unique sequence DADD for 100 and DDAD for 110. Since 101 is the only string left, all its output sequences are unique sequences.

AS0/3 is similar to AS1/2. In AS0/3, $S = \{0, 3\}$ and we detect three input bits a time. When we enter this state, we make sure that at least a 1 bit is withheld. So $I = \{111, 100, 110, 101\}$. R consists of 1) all the 3-tuples generated from row 28 to row 40 in Table I and 2) the cases implied in row 2, i.e., different power output sequences of I starting from State 0. The unique sequences in AS0/3 are listed in the last four rows in Table II.

Once we can detect the input bits with unique sequences in all attack states, we look at how to remove the corresponding power output bits for the detected bits.

C. Committing Detected Bits and Going to Next Attack State

As mentioned earlier, we do not always commit all the detected bits. We may withhold one or two bits. In following attack states, we consider only the strings that start with the withheld bits. For example, in AS1, if the detected string is

111, we only commit the first two 1's and withhold the last 1, which is the leading bit of the input string when we move into AS1/2.

Table III shows how we commit detected bits for all the cases we may have during the attack. The first column in the table is the current attack state. The second column is the detected bits. The third column is bits we need to commit. When a bit is committed, its corresponding power outputs (A's and D's) are removed from the power traces. The fourth column of Table III lists the power outputs of committed bits. The symbols in brackets are not removed from the power traces set because they are the output of the withheld bits in current round of detection. They will be removed when the withheld bits are committed in the next round.

For example, when current attack state is AS1/2 and the detected bits are 111, we only commit the first two 1's and withhold the last one. Starting from AS1/2, 11 generates six different power outputs: ADAD, ADDA, ADD, DAAD, DDA, and DD. When committing 11, we remove these sequences from the power traces. We check each power trace. If it starts with any of ADAD, ADDA, DAAD, and DDA, the outputs of 11 are removed. If a sequence start with ADDD and DDD, we only remove ADD and DD.

TABLE III: COMMITTING DETECTED BITS AND ATTACK STATE TRANSITIONS

Attack state	Bits detected	Bits committed	Power output of committed bits	Next AS
AS0	0	0	D	AS0
	1	1	AD	AS1
AS1	0xx	0	D	AS0
	111	11	ADAD,ADDA,ADD(D)	AS1/2
	100	100	ADADD,ADDD	AS0
	110	1	AD	AS1/2
	101	10	ADAD,ADD	AS0/3
AS1/2	111	11	ADAD,ADDA,ADD(D), DAAD,DDA,DD(D)	AS1/2
	100	100	ADADD,ADDD,DADD	AS0
	110	1	AD,DA,D(D)	AS1/2
	101	10	ADAD,ADD,DAD	AS0/3
AS0/3	111	11	ADAD,ADDA,ADD(D)	AS1/2
	100	100	ADADD,ADDD	AS0
	110	1	AD	AS1/2
	101	10	ADAD,ADD	AS0/3

After we commit some of the detected bits, the power traces are shortened. We can then move into the next attack state and start a new round of detection. The last column of Table III specifies the next attack state for each case.

To summarize, we start from AS0 and detect input bits in the current attack state. Then we commit some of the detected bits, i.e., we append them to the recovered input string and remove their corresponding outputs from power traces. We then go to the next attack state and repeat the process with shortened power traces until all input bits are recovered.

D. Success Rate

Let X be a random variable that indicates the current attack state, Y a random variable that indicates the input bits, and Z the event that the next detected bit is correct. $P[Z|(X=x \cap Y=y)]$ is the success rate in an attack state x when the input string is y , where x can be AS_i ($i=0, 1, 1/2, 0/3$) and y can be 000, 001, 010, ..., or 111. Table IV lists $P[Z|(X=x \cap Y=y)]$ for all four attack states on 3-bit inputs. The probability computations of AS0 and AS1 are the same as those of AS0 and AS1 in [1]. In AS0, we can detect input bit 0 or 1 without mistakes. In AS1, we only detect input string without mistakes when the leading bit is 0 or the input string is 101. In AS1/2, since we withhold at least one bit for each entry and we know the leading bit is 1, we have four cases: 111, 100, 101 and 101. The possibility, $P[Z|(X=AS1/2 \cap Y=y)]$, where $y = 111, 100, \text{ or } 101$, is computed by dividing the number of paths that generate a unique sequence by the total number of possible paths y may take. If, y is the last string after all others are excluded, $P[Z|(X=x \cap Y=y)] = 1$ because y is never misidentified as other strings.

TABLE IV: SUCCESS RATE WITH ONE POWER TRACE

Y \ X	AS0	AS1	AS1/2	AS0/3
0xx	1	1	–	–
100	1	1/2	1/2	3/4
101	1	1	1	1
110	1	1/4	1/4	3/8
111	1	1/4	3/8	1/8

From Table IV, we can calculate the maximal error rate of the attack on Randomized Automaton 2. In AS1, AS1/2, and AS0/3, we may make mistakes when the input string is 100, 110, or 111 because the unique sequences may appear. With a single power trace, the probabilities of not detecting 100, 110, and 111 correctly in AS/1 are 1/2, 3/4, and 3/4, respectively. In AS1/2, the probabilities are 1/2, 3/4, and 5/8, respectively. And they are 1/4, 5/8 and 7/8 in AS0/3. As the number of power traces increases, the probability decreases exponentially. With N power traces, the overall maximum error rate in AS1, AS1/2 or AS0/3 can be calculated as:

$$\text{Max} \left\{ \begin{aligned} & \frac{1}{8} \times \left(\frac{1}{2}\right)^N + \frac{1}{8} \times \left(\frac{3}{4}\right)^N + \frac{1}{8} \times \left(\frac{3}{4}\right)^N, \\ & \frac{1}{4} \times \left(\frac{1}{2}\right)^N + \frac{1}{4} \times \left(\frac{3}{4}\right)^N + \frac{1}{4} \times \left(\frac{5}{8}\right)^N, \\ & \frac{1}{4} \times \left(\frac{1}{4}\right)^N + \frac{1}{4} \times \left(\frac{5}{8}\right)^N + \frac{1}{4} \times \left(\frac{7}{8}\right)^N \end{aligned} \right\}$$

If N is 100, the maximum error rate is about 2^{-21} . If N is 200, the maximum error is about 2^{-40} . So here we can say that our attack is still effective on Randomized Automaton 2.

V. RELATED WORK

Okeya and Takagi proposed a method to attack Randomized Automaton 1 [10]. Their method has two main weaknesses [11]. First, they only focused on Randomized Automaton 1. Second, they overlook the case that one of the points in

addition is the point at infinity. Compared with their algorithms, our attacks are not limited to Randomized Automaton 1. We have shown that our attack on Randomized Automaton 2 is also effective.

VI. CONCLUSIONS AND DISCUSSIONS

In this paper, we presented an attack on randomized ECC algorithms and showed in detail how it can break Randomized Automaton 2 proposed in [5]. Our attack is based on the fact that in every state of the randomized algorithm, an input string generates unique outputs. Although these unique sequences may not appear in every execution, the attackers can exploit them by collecting many power traces. The attack does not require a large number of power traces to achieve a very high success rate.

Our attack can be further extended. Unique sequences are special sequences by which we can identify correct input strings with 100% confidence. One may improve the randomized algorithm and make it unique sequence free. To break such algorithms, we can exploit the distribution of power sequences. As a result, to defeat our attacks, the randomized algorithms have to make sure the distributions of power outputs are the same for all input strings. Our preliminary study shows that such randomized algorithms are difficult to construct and have significant performance overhead.

REFERENCES

- [1] Z. J. Shi and F. Zhang, "New attacks on randomized ECC algorithms," *Proceedings of EITC 2006*, August 2006.
- [2] V. S. Miller, "Use of elliptic curves in cryptography," *Advances in Cryptology, CRYPTO '85*, pp. 417-426, August 1985.
- [3] N. Koblitz, "Elliptic curve cryptosystems," *Mathematics of Computation*, vol. 48, no. 177, pp. 203-209, January 1987.
- [4] S. Vanstone, "ECC holds key to next-gen cryptography," Available: <http://www.us.design-reuse.com/articles/article7409.html>, [Mar 2004].
- [5] E. Oswald and M. Aigner, "Randomized addition-subtraction chains as a countermeasure against power attacks," *Proceedings of CHES 2001*, pp. 39-50, May 2001.
- [6] J. S. Coron, "Resistance against differential power analysis for elliptic curve cryptosystems," *Proceedings of CHES 1999*, pp. 292-302, August 1999.
- [7] A. J. Menezes, "Elliptic curve public key cryptosystems," Kluwer Academic Publishers, 1993.
- [8] P. Kocher, J. Jaffe, and B. Jun, "Differential power analysis," *Proceedings of CRYPTO'99*, pp. 388-397, August 1999.
- [9] F. Morain, and J. Olivos, "Speeding up the computations on an elliptic curve using addition-subtraction chains," *Theoretical Informatics and Applications*, vol. 24, pp. 531-543, 1990.
- [10] K. Okeya, and K. Sakurai, "On insecurity of the side channel attack countermeasure using addition-subtraction chains under distinguishability between addition and doubling," *Proceedings of 7th Australasian Conference on Information Security and Privacy, ACISP 2002*, pp. 420-435, July 2002.
- [11] D. Han, N. S. Chang, S. W. Jung, Y. H. Park, C. H. Kim, and H. Ryu, "Cryptanalysis of the full version randomized addition-subtraction chains," *Proceedings of 8th Australasian Conference on Information Security and Privacy, ACISP 2003*, pp. 67 – 78, July 2003.